Insurance Claims Processing System

Engineering Design Document

Author: Bennett Levine
Date: March 2026

1. Overview

This document outlines the design for a simplified Insurance Claims Processing Platform. The system enables customers to submit insurance claims, allows adjusters to review and process those claims, and includes basic fraud detection mechanisms such as duplicate claim detection.
The platform prioritizes security, traceability, and auditability, which are critical in regulated environments like life insurance. The architecture is designed to be modular, scalable, and integration-ready for downstream underwriting and payment systems.

2. Problem Statement

Insurance claims require structured workflows involving multiple roles:

- Customers submit claims and supporting documentation.
- Adjusters review submitted claims and approve or deny them.
- All actions must be auditable for regulatory compliance.
- Fraud detection mechanisms must identify suspicious behavior.

The system must ensure:

- Secure document handling
- Role-based access control
- Complete audit trails
- Detection of duplicate or suspicious claims
- Integration capability with external systems (payments, underwriting)

3. Goals

Functional Goals

- Customers can submit claims with metadata and documents.
- Adjusters can review claims and approve or deny them.
- System tracks claim status throughout its lifecycle.
- Communication and actions are logged.
- System detects potential duplicate claims.

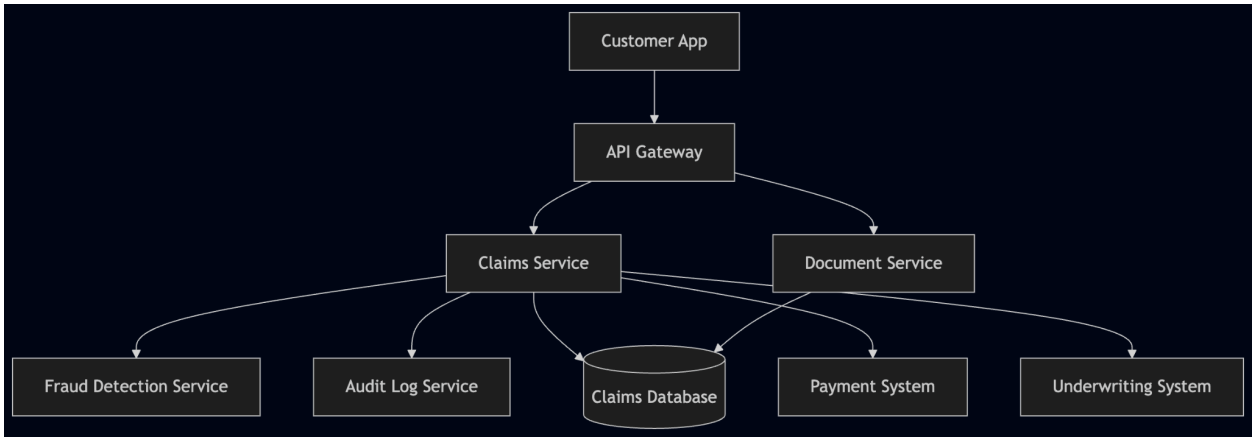
Non-Functional Goals

- Secure handling of sensitive customer data
- Full audit trail of system activity
- Scalable architecture supporting future integrations
- Role-based access control
- Traceability of user actions

4. Assumptions

- Claims volume: 10,000–50,000 claims per day
- Documents may include PDFs or images
- Adjusters access the system via internal applications
- External integrations (payment, underwriting) occur asynchronously
- Fraud detection initially uses rule-based heuristics

5. High-Level Architecture



6. System Components

API Gateway

Responsibilities:

- Authentication
- Request routing
- Rate limiting
- Central entry point for services

This layer ensures security and simplifies integration with future client applications.

Claims Service

Core business logic for managing claims.
Responsibilities:

- Claim submission
- Claim status updates
- Claim retrieval
- Trigger fraud detection checks

- Emit audit logs

This service will be implemented in Part 2 as a REST API.

Document Service

Handles secure storage and retrieval of claim documents.
Responsibilities:

- Secure upload/download
- Encryption at rest
- Access control
- File metadata tracking

Possible implementation:

- Cloud object storage (e.g., S3)
- File reference stored in database

Fraud Detection Service

Responsible for identifying suspicious patterns.
Initial rules may include:

- Duplicate claims for same policy and date
- Excessive claim frequency for a customer
- Identical claim metadata across submissions

Fraud signals are logged and attached to claims for adjuster review.

Audit Log Service

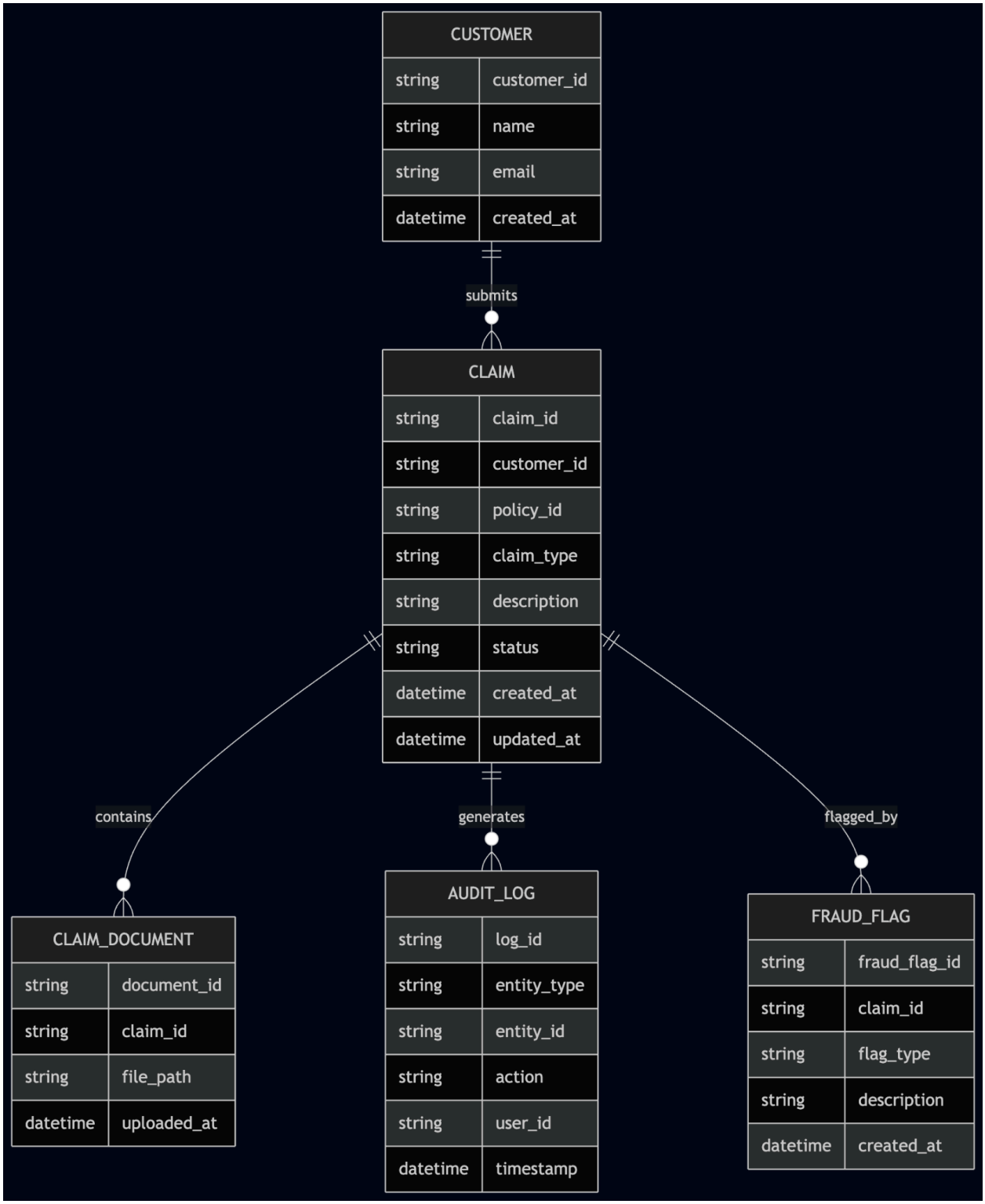
Maintains a tamper-resistant log of all actions.
Examples:

- Claim created
- Claim status changed
- Document uploaded
- Fraud flag triggered

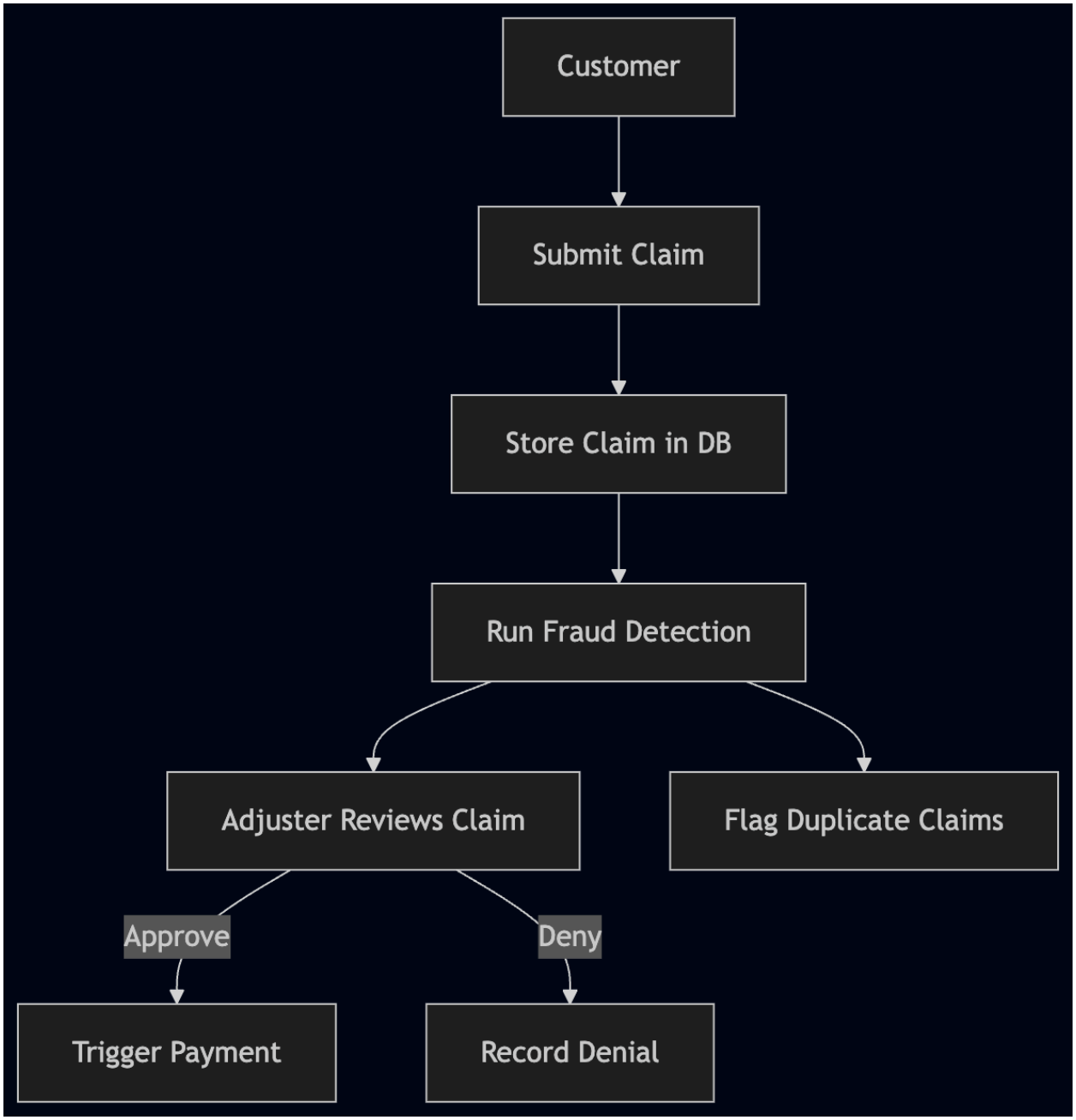
Each record includes:

- User ID
- Timestamp
- Action performed
- Previous and new values

7. Data Model


CUSTOMER
string customer_id
string name
string email
datetime created_at
submits
CLAIM
string claim_id
string customer_id
string policy_id
string claim_type
string description
string status
datetime created_at
datetime updated_at
contains
generates
flagged_by
CLAIM_DOCUMENT
string document_id
string claim_id
string file_path
datetime uploaded_at
AUDIT_LOG
string log_id
string entity_type
string entity_id
string action
string user_id
datetime timestamp
FRAUD_FLAG
string fraud_flag_id
string claim_id
string flag_type
string description
datetime created_at

8. Claims Workflow



9. API Design

Submit Claim

`POST /claims`

Example Request

```
{
  "customerId": "123",
  "policyId": "POL456",
  "claimType": "medical",
  "description": "Hospital visit"
}
```

Retrieve Claims by Customer

`GET /customers/{customerId}/claims`

Update Claim Status

`PATCH /claims/{claimId}`

Example Request

```
{
  "status": "approved",
  "policyId": "POL456",
  "claimType": "medical",
  "description": "Hospital visit"
}
```

Retrieve Duplicate Claims

`GET /claims/duplicates`

Returns claims flagged by the fraud detection service.

10. Fraud Detection Strategy

The system initially uses rule-based fraud detection:
Duplicate detection example:

- Same customer
- Same policy
- Similar claim metadata
- Submitted within a short time window

Future improvements could include:

- Machine learning models
- Behavioral anomaly detection

- Cross-policy fraud analysis

11. Security Considerations

The platform must meet strict security requirements.
Key controls include:

- Role-based access control (RBAC)
- Encryption of documents at rest
- HTTPS for all communications
- Access logging
- Data minimization where possible

Roles include:

- Customer
- Adjuster
- Administrator

12. Scalability Considerations

The system is designed to scale horizontally.
Approaches include:

- Stateless API services
- Independent document storage
- Separate fraud detection service
- Event-driven integrations for payments

Expected performance:

- 50k claims per day
- <200ms API latency for reads
- Horizontal scaling via container orchestration

13. Integration Readiness

The system supports external integrations through APIs and event-based messaging.
Examples:
Payment System

- Trigger payment after claim approval

Underwriting System

- Access historical claims data

Future integrations may use:

- REST APIs
- Message queues

- Event streams

14. Tradeoffs and Alternatives

Monolith vs Microservices

Chosen: Modular service architecture.
Reason:

- Better isolation of fraud detection and document handling
- Easier scaling of high-load components

Rule-Based vs ML Fraud Detection

Chosen: Rule-based initially.
Reason:

- Simpler implementation
- Lower operational complexity
- Adequate for MVP

15. Future Enhancements

Potential improvements include:

- ML-driven fraud detection
- Automated document classification
- Real-time claim processing
- Advanced analytics dashboards
- Policy-level fraud scoring